Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport
la
Lucrarea de laborator Nr. 10

Disciplina: Tehnici de programare

Tema: ” Funcții și excepții ”

A efectuat : st. gr.SI – 261 Talchiv Corina

A verificat : asistent universitar Dumitrașcu Marius

Chișinău – 2026

SARCINA LUCRĂRII

Lucrarea urmărește folosirea funcțiilor și a excepțiilor în Python pe o problemă practică de securitate: analiza unui jurnal de autentificare SSH (auth.log) pentru a găsi adresele IP care încearcă să ghicească parole (atac de tip *brute force*). Obiectivele au fost:

- scrierea de funcții mici, cu un singur rol, care întorc rezultate (return) și pot fi legate între ele;
- tratarea erorilor previzibile (fișier lipsă, linii fără structură, valori invalide);
- crearea unei interfețe în linia de comandă cu argparse;
- verificarea codului cu teste automate (pytest)

Lucrarea A: (parse_auth.py):

Programul funcționează ca o conductă de funcții, în care ieșirea uneia este intrarea următoarei:

```
linie → esec() → (user, ip) → citeste_esecuri() → listă de dicționare → pe_ip() → Counter
```

3.1 Funcția esec(linie)

Analizează o singură linie cu o expresie regulată și întoarce tuplul (user, ip) sau None dacă linia nu este un eșec de autentificare.

TIPAR = re.compile(r'Failed password for (?:invalid user)?(\S+) from (\S+) port')

- re.compile pregătește expresia o singură dată;
- (\S+) este o grupă capturantă (user-ul, apoi IP-ul), accesată cu group(1) și group(2);
- (?:invalid user)? este o grupă necapturantă și opțională: acceptă și rândurile cu invalid user, fără ca acest text să ajungă în rezultat;
- search întoarce None când nu găsește potrivire. Aici None este un semnal (linia nu este un eșec), nu o eroare.

3.2 Funcția citeste_esecuri(cale)

Parcurge fișierul linie cu linie, apelează esec() și adună rezultatele într-o listă de dicționare cu cheile user și ip.

- try/except FileNotFoundError prinde cazul fișierului inexistent și afișează un mesaj clar în loc de traceback; funcția întoarce atunci o listă goală;
- with închide fișierul automat, chiar dacă apare o eroare;
- liniile pentru care esec() întoarce None sunt sărite (if r is not None), deci linia stricată nu oprește programul.

3.3 Funcția pe_ip(esecuri)

Numără eșecurile pentru fiecare IP folosind collections.Counter:

return Counter(e['ip'] for e in esecuri)

Expresia generator parcurge lista, iar Counter întoarce un dicționar specializat de forma IP și număr. Metoda .most_common() îl sortează descrescător.

3.4 Funcția main()

Primește calea fișierului din sys.argv (sau folosește o cale implicită), apelează funcțiile de mai sus și afișează rezumatul. Blocul if __name__ == '__main__': face ca main() să ruleze doar când fișierul este executat direct, nu când este importat.

Rezultat: Esecuri de autentificare: 15 203.0.113.9: 13 198.51.100.7: 2

Lucrarea B: detectorul de brute force (detector.py)

4.1 Funcția suspecti(counts, prag=10)

Întoarce lista de perechi (ip, număr) pentru adresele cu cel puțin prag eșecuri, sortată descrescător.

- prag=10 este un parametru cu valoare implicită, care poate fi schimbat la apel (prag=2);
- validarea if prag < 1: raise ValueError(...) oprește funcția la intrare, cu un mesaj clar; fără ea, un prag de 0 ar marca orice adresă drept suspectă, fără nicio avertizare;
- selecția se face cu sorted(..., key=..., reverse=True) și o listă filtrată.

4.2 Funcția main() și argparse

Interfața în linia de comandă are trei elemente:

- log: argument poziționat obligatoriu (calea fișierului);
- --prag: număr întreg, valoare implicită 10;
- --json: steag (store_true) care schimbă formatul ieșirii.

Eroarea ValueError din suspecti este prinsă în main și transmisă lui p.error(...), astfel că un --prag 0 produce un mesaj curat de utilizare, nu un traceback. Pentru ieșirea JSON se folosește modulul json (json.dumps(raport, indent=2)), iar raportul conține numărul de eșecuri, numărătoarea pe IP și lista suspecților.

Rezultate:

```
Esecuri de autentificare: 15
  203.0.113.9: 13
  198.51.100.7: 2
Suspecti (prag 10):
  203.0.113.9 (13 incercari) BRUTE FORCE
```

CONCLUZII

În concluzie pot spune, că din această lucrare de laborator am înțeles cum o problemă reală (detectarea unui atac brute force) se rezolvă cu funcții mici și clare, legate într-o conductă. Excepțiile au făcut programul robust: fișierul lipsă, linia stricată și pragul invalid sunt tratate controlat, nu prin blocarea programului. Rezultatul final identifică corect adresa 203.0.113.9 (13 încercări eșuate) ca sursă a atacului, iar testele automate confirmă că logica de detecție și validarea funcționează. Abilitățile exersate (regex, Counter, argparse, pytest) sunt reutilizabile în alte sarcini de analiză a jurnalelor.